2026학년도 2학기 AI 서비스 프로젝트

과제 수행계획서

스마트폰 피부 이미지 분류와 맞춤 성분 추천 AI 서비스

파이프라인 개정안

항목	내용
지도교수	사공운 교수님 / 이동인 교수님
팀명	스킨케어
팀 구성원	조장 김우주 / 박성근 · 손승우 · 이현승 · 정도희
개정일	2026년 10월 15일

Swift iOS 앱 → Spring Boot → FastAPI
DenseNet121 이미지 분류 → Qwen2.5 성분 추천

기존 피부 지표 회귀 예측 중심 설계를 현재 구현한 분류· 추천 파이프라인으로 변경한다. AI의 현재 계약과 백엔드· 앱에서 구현할 연동안을 구분해 정리했다.

1 SW 프로젝트 목적 및 필요성

1.1 목적

스마트폰으로 촬영한 피부 이미지와 사용자 설문을 입력받아, 이미지 분류 결과를 참고한 스킨케어 성분과 관리법을 제공한다. 서비스는 Swift 기반 iOS 앱, Spring Boot 백엔드, FastAPI AI 추론 서버로 구성한다.

이미지는 DenseNet121이 건선· 아토피· 여드름· 정상· 주사· 지루의 6개 학습 클래스로 분류한다. FastAPI는 분류 라벨, 점수, 불확실성 및 설문을 Qwen2.5의 입력으로 변환한다. 기존 QLoRA 어댑터를 적용한 Qwen2.5가 성분과 관리법을 생성하고, 서버가 이를 검증· 구조화하여 반환한다.

최종 목표는 이미지 입력부터 분류, 추천 생성, 결과 조회와 이력 관리까지 이어지는 서비스를 완성하는 것이다. 분류 결과는 참고 정보로 제시하고 확정 진단이나 치료 처방으로 표현하지 않는다. 성분 추천 역시 기존 피부 고민 데이터에 기반한 보조 관리 정보로 한정한다.

1.2 필요성

사용자가 사진 분석 결과를 확인하더라도 실제 스킨케어 행동으로 연결하기는 어렵다. 이미지 분류와 설문을 함께 사용하면 결과와 관련된 성분· 관리법을 한 흐름에서 확인할 수 있다. 설명에는 입력에서 확인한 사실과 일반적인 관리 제안을 구분해 제시한다.

프로젝트 일정과 단일 GPU 환경에서는 이미 학습한 분류기와 추천 모델을 연결하고 실제 입력 조건에서 검증하는 작업이 우선이다. 이에 따라 기존의 수분· 탄력· 색소침착· 모공 회귀 예측, 9개 관심영역 분할 및 촬영 환경 보정은 이번 핵심 구현 범위에서 제외한다. 현재 회귀 경로의 임시값은 추천 근거로 사용하지 않는다.

또한 추천 생성에는 분류보다 긴 시간이 필요하다. 백엔드가 작업을 관리하고 앱이 결과를 조회하는 구조를 사용하여, 사용자가 사진 분류 결과를 먼저 확인한 뒤 추천 생성 상태를 이해할 수 있도록 한다.

1.3 차별성

- 이미지 분류의 구조화 출력이 추천 모델의 실제 입력으로 전달된다. 결과 화면에 질환명만 덧붙이는 방식이 아니다.
- 분류 점수가 낮거나 불확실하면 추천을 보류한다. 정상 분류에서 사용자가 선택한 지원 고민이 없으면 불필요한 추천을 생성하지 않는다.
- 분류와 추천의 모델 버전, 처리 시간, 규칙 기반 폴백 사용 여부를 구분해 기록한다. 폴백 답변을 모델 성능 점수에 섞지 않는다.
- 분류 정확도, 추천 품질, API 계약 및 처리 시간을 각각 평가한다. 연결 시험 성공을 질환 판별이나 성분 효능의 검증 결과로 해석하지 않는다.

2 프로젝트 목표

2.1 핵심 기능 및 성능 목표

구분	핵심 기능	구현 범위와 완료 기준
F1	사진과 설문 입력	Swift 앱에서 촬영· 선택, JPEG/PNG 변환, 설문 입력 및 업로드
F2	6종 이미지 분류	DenseNet121의 라벨· 전체 점수· 불확실성 반환
F3	추천 입력 구성	분류 결과를 고민 어휘와 참고 문맥으로 변환하고 설문 결합
F4	성분과 관리법 추천	Qwen2.5와 v2 LoRA를 사용하여 성분· 관리 단계· 설명 생성
F5	작업 상태와 예외 처리	Spring Boot 작업 큐, 결과 조회, 보류· 실패· 재시도 처리
F6	결과와 이력 관리	Swift 결과 화면 및 서버의 구조화 결과· 모델 버전 저장

다음 수치는 개정 계획의 목표다. 독립 평가 전 달성 여부를 단정하지 않으며, 평가 자료와 집계 범위를 함께 보고한다.

평가 대상	목표 또는 판정 기준	평가 방법
이미지 분류	독립 평가셋 Accuracy· Macro F1 각 0.90 이상 목표	클래스별 P/R/F1, 혼동행렬, 정면· 측면 분리 보고
추천 개선 효과	동일 문항에서 원본 Qwen 대비 성분 Top-3 일치율 개선	규칙· 원본· 파인튜닝 비교, 개선폭과 표본 수 공개
추천 내용	사전 검토된 관리 제약 위반 0건 목표	전문가 검토 사례와 반례를 별도 구성
추천 형식	모델 직접 출력 형식 준수율 95% 이상 목표	재시도· 폴백 비율을 분리 집계
입력 거부	준비한 비얼굴 평가셋 거부율 95% 이상 목표	얼굴· 비얼굴· 흐림· 저조도 사례별 평가
분류 지연	준비된 서버에서 단일 요청 p95 3초 이내 목표	이미지 수신 후 분류 반환까지 측정
추천 지연	큐 대기 제외 단일 생성 p95 120초 이내 목표	첫 요청· 재시도· 큐 대기 시간을 별도 기록
동시 요청	10건 시 접수한 작업 누락 0건 목표	직렬 생성· 대기· 완료· 실패 상태의 일관성 검증

기존의 전체 결과 3초 목표는 이미지 분류 단계로 한정한다. 규칙 기반 추천이 기존 평가에서 이미 높은 성분 일치율을 보인 만큼, 단순히 일치율 70%를 넘는 것만으로 파인튜닝 효과를 주장하지 않는다. 추론 설명 유사도는 보조 지표로 사용하고, 설명의 사실성· 제약 준수· 개인화 수준은 별도로 검토한다.

2.2 기타 목표

개인 PC의 RTX 5070 Ti 16GB 환경에서 Qwen은 4-bit, DenseNet은 기본 CPU로 운영한다. AI 학습 중에는 추론 서버를 함께 운영하지 않는다. 원본 업로드 사진은 학습에 재사용하지 않고, 백엔드· AI 서버의 영구 저장과 본문 로그를 방지한다. 얼굴 검출· 입력 품질 확인 및 관리 제약 검증은 사용자 대상 자동 추천 확대 전 보완할 항목이다.

3 프로젝트 구성 요소

3.1 요소 설계 기술

요소 기술 1 이미지 분류

DenseNet121에 Dropout과 6개 클래스 분류층을 적용한 기존 학습 가중치를 사용한다. 서버는 이미지를 RGB로 변환하고 256×256 크기 및 ImageNet 정규화를 적용한다. 최고 점수 라벨과 6개 점수를 반환한다. 이 점수는 임상적으로 보정된 질환 확률이 아니다.

요소 기술 2 분류와 추천의 입력 연결

Qwen의 기존 학습 어휘에 맞춰 아래 규칙으로 고민을 구성한다. 라벨· 점수· 비진단 참고 문맥과 관리 제약을 함께 전달한다. 정상 분류에서는 지원되는 자가응답 고민만 사용하며 입력에 없는 고민이나 측정치를 만들지 않는다.

분류 라벨	Qwen에 전달할 우선 고민
건선 · 아토피 · 지루	민감성(트러블/자극감)
여드름	여드름/뾰루지
주사	붉어짐(홍조), 민감성(트러블/자극감)
정상	사용자가 선택한 지원 고민 사용

분류 점수 0.6 미만 또는 uncertain=true이면 추천을 생성하지 않는다. 0.6은 현 분류기의 운영 임계값이며 검증된 입력 안전성 기준은 아니다. 정상이고 지원 고민이 없는 경우도 추천을 생략한다. 향후 실제 평가셋으로 임계값과 거부 정책을 검증한다.

요소 기술 3 성분 추천 모델과 출력 처리

Qwen/Qwen2.5-7B-Instruct에 기존 rec_sllm_v2 LoRA를 적용한다. 학습에는 성분 추천 데이터와 단계별 설명 라벨을 사용하고, 추론에서는 입력에 근거한 추천 요약과 관리법을 제공한다. Qwen은 사진을 직접 보지 않고 분류 JSON과 설문에서 만든 텍스트를 받는다. 질환 치료 데이터로 재학습한 모델은 아니다.

생성 형식 검사, 재시도, 규칙 기반 폴백, 표현 필터 및 성분· 관리 단계 구조화를 적용한다. used_fallback으로 대체 응답을 구분한다. 필터 통과만으로 성분의 적합성이나 안전성을 입증하지 않으며, 금지 관리법을 권하는 내용은 추가 검증 후 보류하도록 개선한다.

전체 처리 흐름

```
Swift 앱 ── 사진과 설문 ──▶ Spring Boot
Spring Boot ── 사진 ──▶ FastAPI ──▶ DenseNet121
Spring Boot ◀── 분류 JSON ── FastAPI
Spring Boot 작업 큐 ── 분류 JSON과 설문 ──▶ FastAPI ──▶ Qwen2.5
Swift 앱 ◀── 결과 조회 ── Spring Boot ◀── 추천 JSON ── FastAPI
```

Swift는 Spring Boot만 호출한다. FastAPI의 내부 인증 토큰은 앱에 포함하지 않는다. 분류를 먼저 반환하는 2단계 방식을 기본 연동안으로 채택하며, 한 번에 실행하는 AI API는 통합 시험과 시연에 활용한다.

3.2 구현 사양

계층	기술과 구성	담당 기능
클라이언트	Swift 기반 iOS 앱, URLSession· Codable	촬영· 설문· 업로드· 작업 조회· 결과 표시
백엔드	Spring Boot REST API, 서버 작업 큐	인증· 입력 검증· AI 호출· 상태 관리· 이력 저장
데이터베이스	PostgreSQL 사용안	사용자· 설문· 분류· 추천· 작업 상태· 모델 버전
AI 서버	Python 3.11, FastAPI· Uvicorn· Pydantic	내부 API· 분류· 추천· 스키마 검증· 오류 응답
분류 추론	PyTorch· torchvision· Pillow, DenseNet121 CPU	6종 분류 및 점수 반환
추천 추론	Transformers· PEFT· bitsandbytes, Qwen2.5 GPU	4-bit 추론 및 v2 어댑터 적용
운영	개인 GPU PC와 백엔드 서버, HTTPS 연결	내부 토큰· 환경변수· 모델 준비 상태 점검

데이터베이스 선택, 배포 방식 및 Spring Boot의 세부 라이브러리는 연동 구현 시 확정한다. RTX 5070 Ti에서는 현재 확인한 CUDA 12.8 계열 PyTorch· torchvision 조합을 기준으로 환경을 고정한다. ONNX 변환은 현재 파이프라인의 필수 구성으로 주장하지 않는다.

현재 FastAPI에 /infer/disease, /infer/disease-recommend, /infer/disease-analyze가 구현되어 있다. Spring Boot의 작업 API와 Swift 화면 변경은 이 문서를 기준으로 진행할 개발 항목이다. 기존 /infer/analyze의 skin.is_stub=true 값은 실제 측정치로 표시하거나 추천에 사용하지 않는다.

3.3 필요 데이터 리스트 및 수집 경로

데이터	출처와 현재 상태	활용 및 추가 작업
안면부 피부질환 이미지 합성 데이터	AI Hub 71863, 기존 DenseNet 가중치· 평가 산출물 활용	원본 이미지· 정답 평가셋 확보 후 재평가
스킨케어 성분 효능 추천 데이터	AI Hub 71886, 정제 데이터 및 학습된 v2 어댑터 활용	원본 대비 추천 성능 비교, 성분명 정규화
관리 제약과 검토 근거	전문가 검토 자료를 추가 준비할 계획	질환별 회피 조건과 설명의 적합성 평가
실제 사진 평가 자료	동의와 사용 권한을 확보한 사진· 정답	기기· 각도· 피부 조건별 실제 일반화 평가
비얼굴 및 품질 반례	단색· 잡음· 비얼굴· 흐림· 저조도 입력	입력 거부와 재촬영 안내 검증

학습 데이터 및 파생 자료는 각 이용 조건에 따라 관리한다. 원본 사진과 데이터셋을 Git 저장소에 올리지 않는다. 추천 모델 실행에는 별도 배포하는 LoRA 어댑터와 성분 사전이 필요하다. 데이터 접근 없이 연동하는 팀원에게는 명시적으로 표시한 시험용 JSON과 규칙 기반 mock 모드를 제공한다.

3.4 솔루션 참고 기준

분류 구조와 전처리는 팀의 DenseNet 원본 브랜치를 기준으로 재현한다. 추천 모델 구성은 Qwen 모델 카드와 PEFT 문서를, API와 클라이언트 통신은 FastAPI· Spring· Apple 공식 문서를 참고한다. 구체적인 출처와 로컬 검증 근거는 5절에 정리한다.

4 프로젝트 운영 계획

4.1 팀 구성 및 역할

구성원	담당 역할	주요 인계 산출물
김우주 조장	Qwen 학습· 평가, FastAPI, 모델 통합 및 일정 관리	AI 계약· 실행 가이드· 비교 평가· 발표자료
박성근	DenseNet 분류기, 전처리, 실제 사진 성능 평가	가중치· 클래스 정의· 분류 평가· 오류 분석
손승우	요구사항· 계획서· 회의록, 개인정보· 표현 검토	변경 문서· 검토 기록· 최종 보고서
이현승	Spring Boot API· DB· 작업 큐와 AI 연동	분석 접수· 상태 조회 API· 오류 처리· 저장 구조
정도희	Swift iOS 앱과 사용자 흐름, 백엔드 연동	촬영· 설문· 대기· 보류· 결과· 이력 화면

AI 서버는 조장 개인 PC에서 운영한다. 백엔드· DB의 배포 위치와 앱 배포 방식은 팀에서 확정한다. 문서· 계약 변경 시 AI, 백엔드, 프론트 담당자가 같은 버전의 요청· 응답 예시를 사용한다.

4.2 프로젝트 운영 계획

기존 양식의 학기 일정 틀을 유지하되, 10월 15일 이후의 작업을 새 파이프라인에 맞춰 조정한다. 지나간 기간의 모든 항목이 완료되었다는 뜻은 아니다. 중간 발표 10월 20일은 첨부 계획서 기준이며 수업 공지에 맞춰 최종 확인한다.

단계	일정	추진 내용과 담당	산출물
자료와 기본 설계	9/15~10/5	데이터· 환경· 초기 API 정리, 전원	초기 계획· 설계 자료
모델과 AI 연결	10/6~10/15	DenseNet· Qwen 연결 및 로컬 시험, 김우주· 박성근	AI API· 시험 기록
계약 확정과 중간 시연	10/16~10/20	Swift· Spring DTO 및 상태 합의, 전원	개정 계획· 연동 예시· 중간 발표
서비스 연동과 품질 보완	10/21~11/9	작업 큐· 앱 화면· 입력 거부· 관리 제약 보완	통합 시제품
기본 기능 시험	11/10~11/24	실제 사진· 반례· 부하· 재시도 검증, 전원	기능· 품질 시험 결과
기능 완료와 안정화	11/25~12/1	오류 수정· 배포 점검· 발표 준비, 전원	시연 버전· 발표자료
성능 분석과 보고	12/2~12/8	목표 대비 결과· 한계· 개선안 정리	최종 보고서

4.3 협업 및 변경 관리

GitHub 조직의 AI· 백엔드· 프론트 저장소를 분리 운영한다. API 필드와 상태 변경은 예시 JSON과 계약 시험을 함께 갱신한다. AI에는 가중치· 사전· 어댑터의 버전을 기록하고, 백엔드는 분석별 버전을 저장한다. 모델 학습과 서버 운영 시간을 분리하고 처리 중인 작업의 종료· 실패 상태를 남긴다.

5 검증 계획과 참고문헌

5.1 현재 확인한 결과와 한계

2026년 10월 15일, 이 PC에서 DenseNet121과 Qwen2.5 v2를 실제 로드한 HTTP 연결 시험을 수행했다. 전체 26건은 예상 HTTP 상태와 일치했고, 시험용 분류 JSON 10건은 규칙 폴백 없이 추천을 생성했다. 생성 시간은 중앙값 43.33초, 최대 50.04초였다. 이는 실제 환자 사진의 질환 정확도 또는 추천 내용의 적합성 통과율이 아니다.

비얼굴 대조군 6건 중 4건이 높은 분류 점수로 수락되었다. 일부 추천은 입력의 관리 제약을 따르지 않았고, 성분 설명이 없거나 파손된 항목도 있었다. 현재 연결 구현을 사용자 대상 자동 질환 추천의 검증 완료로 간주하지 않는다. 실제 사진 평가는 자료 확보 후 별도로 수행한다.

기존 분류 브랜치의 test 기록은 1,080건에서 Accuracy 94.91%, Macro F1 94.89%다. 정면 90.74%, 측면 99.07%로 차이가 있으며, 이는 원본 브랜치의 보고값이다. 이번 통합 시험에서 재측정한 결과와 구분해 발표한다.

5.2 시험 절차

- 사진· 라벨· 피험자 단위 중복을 확인하고 학습과 평가를 분리한다. 6종 균형, 정면· 측면 및 촬영 조건별 성능을 보고한다.
- 규칙· 원본 Qwen· 파인튜닝 Qwen을 같은 고정 평가 문항으로 비교한다. 원시 모델 출력, 재시도 및 폴백 결과를 분리한다.
- 불확실· 정상 무고민· 비얼굴· 잘못된 JSON· 업로드 오류· AI 중단· 시간 초과를 계약 시험에 포함한다.
- 성분 설명 누락, 근거 없는 부위· 원인 단정, 관리 제약 위반을 수동 검토한다. 허용 여부가 확인되지 않은 내용은 전문가 검토 대상으로 남긴다.
- 앱에서 접수· 분류· 대기· 추천 완료· 보류· 실패 화면을 확인한다. 10건 동시 접수 시 작업 누락과 중복 생성을 점검한다.

5.3 참고문헌 및 출처

[1] AI Hub, 안면부 피부질환 이미지 합성 데이터, dataSetSn 71863. 데이터 설명· 이용 조건은 수령 시 재확인한다.

https://www.aihub.or.kr/aihubdata/data/view.do?dataSetSn=71863

[2] AI Hub, 스킨케어 성분 효능 추천 데이터, dataSetSn 71886. 기존 프로젝트의 학습 데이터 출처.

https://www.aihub.or.kr/aihubdata/data/view.do?dataSetSn=71886

[3] Qwen, Qwen2.5-7B-Instruct Model Card.

https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[4] Hugging Face, PEFT Quantization.

https://huggingface.co/docs/peft/main/developer_guides/quantization

[5] FastAPI 공식 문서 / Spring Framework WebClient / Apple URLSession.

https://fastapi.tiangolo.com/

https://docs.spring.io/spring-framework/reference/web/webflux-webclient.html

https://developer.apple.com/documentation/foundation/urlsession

[6] 팀 저장소의 분류 원본 커밋 3ea9506bf72f096b3e2bd545f241a8c57fcbfe46, model/results/baseline_s2/eval_test/metrics.json 및 docs/disease-pipeline-quality-20261015.md. 외부 데이터셋 이미지는 저장소에 포함되어 있지 않다.

부록 A AI 요청과 응답 계약

A.1 이미지 분류 요청

이 부록의 /infer/* 경로와 snake_case 필드는 현재 FastAPI 구현 기준이다. Spring Boot만 내부 주소로 호출하며 X-Internal-Token 헤더를 사용한다. 앱은 사용자 인증으로 Spring Boot에 접속한다.

```
POST /infer/disease
Content-Type: multipart/form-data
X-Internal-Token: 서버에 보관한 값
image: JPEG 또는 PNG 파일 1개, 최대 10MiB
```

분류 응답 예시는 아래와 같다. 수치는 구조 설명용이며 실제 사진 판정 결과가 아니다. confidence를 질환 확률로 표시하지 않는다.

```
{
  "label": "아토피",
  "label_en": "atopic",
  "confidence": 0.9,
  "uncertain": false,
  "probabilities": {
    "건선": 0.02, "아토피": 0.9, "여드름": 0.02,
    "정상": 0.02, "주사": 0.02, "지루": 0.02
  },
  "model_version": "densenet121_s2_3ea9506",
  "inference_ms": 45
}
```

A.2 성분 추천 요청

```
POST /infer/disease-recommend
Content-Type: application/json
X-Internal-Token: 서버에 보관한 값
```

필드	자료형과 필수 여부	처리 기준
disease	위 분류 객체 전체, 필수	라벨만 넘기지 않고 6개 점수도 전달
survey	객체, 필수	gender 필수, age 또는 ageBand 및 skinType 권장
survey.selfReportedConcerns	문자열 배열, 선택	지원되는 자가응답 고민, 없으면 빈 배열
survey.note	문자열, 선택	사용자 추가 요청, 최대 300자
metrics	객체 또는 null, 선택	현재 연동은 null 또는 생략, 임시 측정값 금지

설문은 gender: female/male, age: 10~100 정수, skinType: dry/normal/oily/combination을 사용한다. ageBand의 중간값 변환 등 가정은 응답의 input_notes로 확인한다.

```
{
  "disease": {
    "label": "아토피", "label_en": "atopic", "confidence": 0.9, "uncertain": false,
    "probabilities": {
      "건선": 0.02, "아토피": 0.9, "여드름": 0.02,
      "정상": 0.02, "주사": 0.02, "지루": 0.02
    },
    "model_version": "densenet121_s2_3ea9506", "inference_ms": 45
  },
  "survey": {"gender": "female", "age": 30, "skinType": "dry",
             "selfReportedConcerns": []},
  "metrics": null
}
```

A.3 추천 응답과 표시 기준

FastAPI의 성공 응답은 아래 필드를 갖는다. JSON의 성분명· 문장· 시간은 구조 설명용 예시이며 검증된 효능이나 현재 모델 출력의 인용이 아니다. disease에는 A.1의 전체 분류 객체가 들어간다.

실행 가능한 요청과 기록된 응답은 docs/fixtures/disease-pipeline/에 제공한다. fixture 입력은 시험용 JSON이며 실제 환자 판정이 아니다.

```
{
  "disease": {
    "label": "아토피", "label_en": "atopic",
    "confidence": 0.9, "uncertain": false,
    "probabilities": {
      "건선": 0.02, "아토피": 0.9, "여드름": 0.02,
      "정상": 0.02, "주사": 0.02, "지루": 0.02
    },
    "model_version": "densenet121_s2_3ea9506", "inference_ms": 45
  },
  "recommendation_status": "generated",
  "recommendation": {
    "skin_type": "건성",
    "concerns": ["민감성(트러블/자극감)"],
    "ingredients": [
      { "name": "성분명 예시", "purpose": null, "priority": 1 }
    ],
    "routine": [
      { "step": 1, "time": "daily", "action": "관리법 설명 예시" }
    ],
    "reasoning": ["입력과 관련된 추천 설명 예시"],
    "answer": "사용자에게 보여줄 추천 요약 예시",
    "disclaimer": "참고 정보 안내",
    "filtered": false,
    "used_fallback": false,
    "input_notes": [],
    "model_version": "rec_sllm_v2",
    "inference_ms": 43000
  },
  "guidance": ["이미지 분류 참고 결과이며 의학적 진단이 아닙니다."]
}
```

recommendation_status	recommendation 값	앱에서 보여줄 내용
generated	추천 객체	성분· 관리 단계· 요약과 안내문
uncertain	null	판단 보류와 재촬영· 상담 안내
no_concern	null	자동 추천할 고민 없음, 설문 선택 안내

ingredients는 3개 고정이 아니므로 배열 길이에 맞춰 표시한다. purpose는 null일 수 있어 설명이 없는 항목은 성분명만 표시하고 임의 효능을 만들지 않는다. routine.time은 morning/evening/daily/weekly/anytime 중 하나다. reasoning은 생성된 설명으로 취급하며, 검증된 인과 근거로 표시하지 않는다.

filtered는 표현 필터가 개입했다는 표시이며 검증 완료 표시가 아니다. used_fallback=true는 규칙 기반 대체 답변이다. 모델 버전과 처리 시간은 운영 기록에 사용한다. 화면에 모델 점수를 표시한다면 0~1 값을 백분율로 변환하고 반드시 ‘모델 분류 점수’라고 표기한다.

A.4 한 번에 실행하는 API

POST /infer/disease-analyze에 multipart image와 survey JSON 문자열을 보내면 분류와 추천을 한 번에 실행한다. 응답은 A.3과 같다. 실제 생성까지 연결하는 통합 시험에 사용하며, 서비스 앱의 기본 흐름은 부록 B의 비동기 2단계 방식으로 구현한다.

부록 B Spring Boot와 Swift 연동 구현안

B.1 분석 접수와 작업 처리

아래 /api/v1/* 경로, camelCase 필드 및 작업 상태는 백엔드· 앱을 위해 제안하는 새 계약이다. 현재 FastAPI의 구현 완료 API로 혼동하지 않는다. 이현승· 정도희 담당자가 실제 서버와 앱에 반영하며 변경 시 양측 DTO를 함께 갱신한다.

1. Swift는 촬영 파일을 JPEG/PNG로 만들고 POST /api/v1/analyses에 image 파일과 survey JSON 문자열을 전송한다. HEIC 원본은 그대로 보내지 않는다.
2. Spring Boot는 사용자 인증과 소유권, 파일 형식· 용량을 검증한 뒤 FastAPI /infer/disease로 이미지를 보낸다. 사진은 영구 저장하지 않고 분류 종료 후 참조를 해제한다.
3. Spring Boot는 분류 JSON과 설문, 분석 ID를 저장한다. 추천 가능한 경우 작업을 큐에 넣고 HTTP 202와 초기 결과를 반환한다.
4. 작업자는 저장한 전체 분류 JSON과 설문으로 /infer/disease-recommend를 호출한다. 분류기 저점수· 불확실 조건에서는 이 API로 보류 안내만 받아 저장하고 GPU 생성 작업은 만들지 않는다.
5. 정상 분류의 추천 여부와 최종 recommendationStatus는 AI 응답을 따른다. AI의 고민 매핑 규칙을 앱이나 백엔드에 별도로 복제하지 않는다.
6. Swift는 GET /api/v1/analyses/{analysisId}를 조회한다. 완료· 보류· 실패 시 조회를 종료하고 앱 재진입 시 같은 ID로 결과를 다시 확인한다.

최초 접수의 권장 응답 구조는 다음과 같다. disease는 A.1의 객체를 Spring DTO에서 camelCase로 변환한다.

```
{
  "analysisId": "분석 ID",
  "status": "QUEUED",
  "disease": { "분류 객체": "설명용 생략" },
  "recommendationStatus": null,
  "recommendation": null,
  "guidance": [],
  "error": null
}
```

status	의미	Swift 처리
QUEUED	접수 후 생성 대기	분류 참고 결과와 대기 안내
GENERATING	AI 추천 호출 중	진행 표시, 중복 접수 방지
COMPLETED	추천 생성 완료	추천 및 공통 안내 표시
WITHHELD	uncertain 또는 no_concern	추천 없이 사유와 다음 행동 표시
FAILED	통신· 생성· 저장 실패	오류 안내와 허용된 재시도

최종 조회 응답은 같은 형태를 유지한다. status는 백엔드 작업 상태이고 recommendationStatus는 AI의 generated/uncertain/no_concern 값이다. 보류를 오류로 처리하거나 빈 추천 객체로 대체하지 않는다. 저점수 보류 결과를 접수 단계에서 확정할 수 있으면 HTTP 200과 WITHHELD를 반환해도 된다.

B.2 시간 제한과 대기열

구간	초기 설정안	운영 기준
Spring → 이미지 분류	읽기 제한 20초	분류 목표 3초와 통신 실패 제한을 구분
FastAPI 추천 생성	현재 120초	생성 슬롯 획득 후 실행 시간 기준
Spring → 추천 호출	130초 이상	앱 요청과 분리한 작업자에서 호출
Spring 작업자	생성 동시 1건, 대기 최대 10건 안	초과 접수 제한, DB에 상태 기록
Swift 상태 조회	처음 3분은 3초, 이후 10초 간격 안	각 GET은 짧게 종료, 단일 장기 요청 불필요
사용자 대기 안내	150초 이후 지연 안내, 최대 30분 조회 안	큐 대기는 생성 시간과 별도, 종료 후 이력 재조회

위 백엔드· 앱 수치는 단일 GPU 운용을 위한 초기 설계안이다. 10건을 동시에 생성한다는 뜻이 아니다. 부하 시험 후 큐 크기와 전체 작업 만료 시간을 함께 조정한다. 20초 제한을 추천 호출에 적용하거나 앱이 30초 후 무조건 실패 처리하면 현재 모델의 정상 응답도 받지 못한다.

AI가 시간 초과를 반환해도 내부 GPU 연산은 끝날 때까지 실행될 수 있다. 즉시 같은 요청을 중복 생성하지 않는다. 백엔드는 추천 재시도 시 기존 분류 결과를 재사용하고 작업 ID로 중복 실행을 막는다. 앱 종료· 화면 이탈은 서버 작업 취소와 동일하게 취급하지 않는다.

B.3 오류와 화면 대응

AI HTTP 및 코드	의미	백엔드 및 앱 대응
401 E1001	내부 토큰 오류	서버 설정 점검, 앱에 토큰 입력을 요구하지 않음
413 E2003	이미지 용량 초과	압축 또는 재선택 안내
415 E2004	JPEG/PNG 외 형식	iOS 변환 및 지원 형식 안내
422 E2002 / E4002	디코딩 실패 또는 요청 검증 실패	재촬영· 입력 확인, 개발자는 DTO 점검
503 E3001 / E3003	분류 비활성 또는 대기열 초과	서버 준비· 혼잡 안내, 제한된 재시도
504 E3002	AI 실행 시간 초과	작업 실패 기록, 중복 생성 방지

AI 오류 본문은 success:false와 error:{code,message,detail} 구조다. 백엔드는 내부 detail· 토큰· 원본 입력을 앱과 로그에 그대로 노출하지 않는다. 얼굴 미검출 전용 오류는 아직 구현되지 않았으므로 구현 완료로 표시하지 않는다.

B.4 담당자별 코드 변경 범위

Spring Boot는 분류· 추천 DTO, snake_case에서 camelCase로의 변환, 내부 토큰, AI 클라이언트, 작업 큐· 조회 API, nullable 추천 저장 및 모델 버전 기록을 구현한다. Swift는 Codable DTO, multipart 업로드, HEIC 변환, 상태 조회, 추천 보류· 실패 화면, purpose null과 가변 배열 처리를 반영한다.

FastAPI는 현재 계약과 예시를 유지하며 입력 거부· 관리 제약· 성분 설명 구조화를 보완한다. 실제 사진 평가 자료가 준비되면 이미지부터 최종 화면까지 다시 시험한다. 세 계층의 기본 인계 자료는 실행 가이드, 요청· 응답 fixture 및 로컬 품질 점검 보고서로 통일한다.